Object-Oriented Programming with C++
Unit 2: Inheritance — Practical Code Book

Course: Object-Oriented Programming with C++

Course Code: ADPC303

Programme: S.Y. B.Tech. Artificial Intelligence and Data Science

Language Standard: C++17 or later

How to Use This Code Book
For each Concept:

1. Read the aim.
2. Understand the concept.
3. Type the program yourself.
4. Compile and run it.
5. Compare the output.
6. Modify one value or feature.
7. Answer the viva questions.

Compilation command

```
g++ -std=c++17 filename.cpp -o program
./program
```

On Windows using MinGW:

```
g++ -std=c++17 filename.cpp -o program.exe
program.exe
```

Index

Sr. No.	Title	Main concept
1	Basic single inheritance	Base and derived classes
2	Protected member access	protected access specifier
3	Public versus private inheritance	Inheritance modes
4	Multilevel inheritance	Three-level hierarchy
5	Hierarchical inheritance	One base, multiple derived classes
6	Multiple inheritance	Two base classes
7	Multiple-inheritance ambiguity	Scope-resolution operator
8	Constructor and destructor order	Object lifecycle
9	Parameterized base constructor	Initializer list
10	Function overriding	virtual and override
11	Abstract class	Pure virtual function
12	Virtual base class	Diamond inheritance
13	Friend class	Special access permission
14	Nested class	Class inside another class
15	Mini-project: Vehicle rental	Integrated inheritance
16	Mini-project: Employee payroll	Abstract base and overriding

Concept 1: Basic Single Inheritance

Aim

To implement single inheritance using `Person` as a base class and `Student` as a derived class.

Concept

A derived class receives accessible members from a base class. `Student` is a `Person` is an "is-a" relationship.

Program

```
#include <iostream>
#include <string>
#include <utility>

class Person {
protected:
    std::string name;

public:
    explicit Person(std::string personName) : name(std::move(personName)) {}

    void displayName() const {
        std::cout << "Name: " << name << '\n';
    }
};

class Student : public Person {
private:
```

```
    int rollNumber;

public:
    Student(std::string studentName, int roll)
        : Person(std::move(studentName)), rollNumber(roll) {}

    void displayStudent() const {
        displayName();
        std::cout << "Roll Number: " << rollNumber << '\n';
    }
};

int main() {
    Student student("Amit", 101);
    student.displayStudent();
    return 0;
}
```

Output

```
Name: Amit
Roll Number: 101
```

Viva questions

1. Which class is the base class?
2. Which class is the derived class?
3. Why is public inheritance used here?

Concept 2: Protected Member Access

Aim

To demonstrate how a derived class can access a protected base-class member.

Program

```
#include <iostream>
#include <string>
```

```
#include <utility>

class Employee {
protected:
    std::string name;

public:
    explicit Employee(std::string employeeName) : name(std::move(employeeName)) {}
};

class Developer : public Employee {
private:
    std::string language;

public:
    Developer(std::string employeeName, std::string programmingLanguage)
        : Employee(std::move(employeeName)), language(std::move(programmingLanguage)) {}

    void display() const {
        std::cout << "Developer: " << name << '\n';
        std::cout << "Language: " << language << '\n';
    }
};

int main() {
    Developer developer("Neha", "C++");
    developer.display();
    return 0;
}
```

Output

```
Developer: Neha
Language: C++
```

Key learning

A protected member can be accessed inside the base class and its derived classes, but not directly from outside code.

Concept 3: Public versus Private Inheritance

Aim

To observe the effect of public and private inheritance on accessibility.

Program

```
#include <iostream>

class Base {
public:
    void show() const {
        std::cout << "Base public function\n";
    }
};

class PublicDerived : public Base {
};

class PrivateDerived : private Base {
public:
    void callBaseShow() const {
        show();
    }
};

int main() {
    PublicDerived publicObject;
    publicObject.show();

    PrivateDerived privateObject;
    privateObject.callBaseShow();
```

```
    // privateObject.show(); // Error: show() is private through private inheritance.

    return 0;
}
```

Output

```
Base public function
Base public function
```

Key learning

- In public inheritance, public base members stay public through the derived object.
- In private inheritance, public and protected base members become private in the derived class.

Concept 4: Multilevel Inheritance

Aim

To implement multilevel inheritance using Person → Employee → Manager.

Program

```
#include <iostream>
#include <string>
#include <utility>

class Person {
protected:
    std::string name;

public:
    explicit Person(std::string personName) : name(std::move(personName)) {}

    void showPerson() const {
        std::cout << "Name: " << name << '\n';
    }
```

```
};

class Employee : public Person {
protected:
    int employeeId;

public:
    Employee(std::string employeeName, int id)
        : Person(std::move(employeeName)), employeeId(id) {}

    void showEmployee() const {
        std::cout << "Employee ID: " << employeeId << '\n';
    }
};

class Manager : public Employee {
private:
    int teamSize;

public:
    Manager(std::string managerName, int id, int size)
        : Employee(std::move(managerName), id), teamSize(size) {}

    void showManager() const {
        showPerson();
        showEmployee();
        std::cout << "Team Size: " << teamSize << '\n';
    }
};

int main() {
    Manager manager("Ravi", 501, 8);
    manager.showManager();
    return 0;
```

```
}
```

Output

```
Name: Ravi
Employee ID: 501
Team Size: 8
```

Concept 5: Hierarchical Inheritance

Aim

To implement hierarchical inheritance using `Vehicle` as a common base class.

Program

```
#include <iostream>
#include <string>
#include <utility>

class Vehicle {
protected:
    std::string registrationNumber;

public:
    explicit Vehicle(std::string registration)
        : registrationNumber(std::move(registration)) {}

    void start() const {
        std::cout << "Vehicle " << registrationNumber << " started\n";
    }
};

class Car : public Vehicle {
public:
    explicit Car(std::string registration) : Vehicle(std::move(registration)) {}

    void openBoot() const {
```

```
        std::cout << "Car boot opened\n";
    }
};

class Bike : public Vehicle {
public:
    explicit Bike(std::string registration) : Vehicle(std::move(registration)) {}

    void helmetReminder() const {
        std::cout << "Please wear a helmet\n";
    }
};

int main() {
    Car car("MH12AB1234");
    Bike bike("MH12CD5678");

    car.start();
    car.openBoot();

    bike.start();
    bike.helmetReminder();

    return 0;
}
```

Output

```
Vehicle MH12AB1234 started
Car boot opened
Vehicle MH12CD5678 started
Please wear a helmet
```

Concept 6: Multiple Inheritance

Aim

To demonstrate multiple inheritance using academic and sports records.

Program

```
#include <iostream>

class Academic {
protected:
    int academicMarks;

public:
    explicit Academic(int marks) : academicMarks(marks) {}

    void showAcademic() const {
        std::cout << "Academic Marks: " << academicMarks << '\n';
    }
};

class Sports {
protected:
    int sportsMarks;

public:
    explicit Sports(int marks) : sportsMarks(marks) {}

    void showSports() const {
        std::cout << "Sports Marks: " << sportsMarks << '\n';
    }
};

class Student : public Academic, public Sports {
public:
    Student(int academic, int sports)
        : Academic(academic), Sports(sports) {}
```

```
    void showTotal() const {
        std::cout << "Total Marks: " << academicMarks + sportsMarks << '\n';
    }
};

int main() {
    Student student(80, 15);
    student.showAcademic();
    student.showSports();
    student.showTotal();
    return 0;
}
```

Output

```
Academic Marks: 80
Sports Marks: 15
Total Marks: 95
```

Concept 7: Resolving Multiple-Inheritance Ambiguity

Aim

To resolve ambiguity when two base classes contain methods with the same name.

Program

```
#include <iostream>

class Academic {
public:
    void display() const {
        std::cout << "Academic information\n";
    }
};

class Sports {
public:
```

```
    void display() const {
        std::cout << "Sports information\n";
    }
};

class Student : public Academic, public Sports {
public:
    void displayAll() const {
        Academic::display();
        Sports::display();
    }
};

int main() {
    Student student;

    student.Academic::display();
    student.Sports::display();
    student.displayAll();

    return 0;
}
```

Output

```
Academic information
Sports information
Academic information
Sports information
```

Key learning

Use the scope-resolution operator to specify the required base-class member:

```
student.Academic::display();
student.Sports::display();
```

Concept 8: Constructor and Destructor Order

Aim

To observe the construction and destruction order of a derived object.

Program

```
#include <iostream>

class Base {
public:
   Base() {
      std::cout << "Base constructor\n";
   }

   ~Base() {
      std::cout << "Base destructor\n";
   }
};

class Derived : public Base {
public:
   Derived() {
      std::cout << "Derived constructor\n";
   }

   ~Derived() {
      std::cout << "Derived destructor\n";
   }
};

int main() {
   Derived object;
   return 0;
}
```

Output

```
Base constructor
Derived constructor
Derived destructor
Base destructor
```

Rule

```
Creation:    Base constructor → Derived constructor
Destruction: Derived destructor → Base destructor
```

Concept 9: Parameterized Base Constructor

Aim

To initialize a parameterized base class from a derived-class constructor.

Program

```
#include <iostream>
#include <string>
#include <utility>

class Person {
protected:
    std::string name;

public:
    explicit Person(std::string personName) : name(std::move(personName)) {}
};

class Student : public Person {
private:
    int rollNumber;

public:
    Student(std::string studentName, int roll)
```

```
    : Person(std::move(studentName)), rollNumber(roll) {}

  void display() const {
    std::cout << "Name: " << name << '\n';
    std::cout << "Roll Number: " << rollNumber << '\n';
  }
};

int main() {
  Student student("Kiran", 24);
  student.display();
  return 0;
}
```

Output

```
Name: Kiran
Roll Number: 24
```

Concept 10: Function Overriding

Aim

To override a virtual member function in a derived class.

Program

```
#include <iostream>

class Vehicle {
public:
  virtual void move() const {
    std::cout << "Vehicle is moving\n";
  }

  virtual ~Vehicle() = default;
};
```

```
class Car : public Vehicle {
public:
    void move() const override {
        std::cout << "Car moves on roads\n";
    }
};

class Boat : public Vehicle {
public:
    void move() const override {
        std::cout << "Boat moves on water\n";
    }
};

int main() {
    Car car;
    Boat boat;

    car.move();
    boat.move();
    return 0;
}
```

Output

```
Car moves on roads
Boat moves on water
```

Key learning

- `virtual` in the base class enables run-time polymorphic behavior.
- `override` in the derived class helps the compiler verify the intended override.

Concept 11: Abstract Class

Aim

To create an abstract base class using a pure virtual function.

Program

```
#include <iostream>

class Shape {
public:
   virtual double area() const = 0;
   virtual ~Shape() = default;
};

class Rectangle : public Shape {
private:
   double length;
   double width;

public:
   Rectangle(double givenLength, double givenWidth)
      : length(givenLength), width(givenWidth) {}

   double area() const override {
      return length * width;
   }
};

class Circle : public Shape {
private:
   double radius;

public:
   explicit Circle(double givenRadius) : radius(givenRadius) {}

   double area() const override {
      return 3.141592653589793 * radius * radius;
   }
```

```
};

int main() {
    Rectangle rectangle(5.0, 3.0);
    Circle circle(2.0);

    std::cout << "Rectangle Area: " << rectangle.area() << '\n';
    std::cout << "Circle Area: " << circle.area() << '\n';
    return 0;
}
```

Output

```
Rectangle Area: 15
Circle Area: 12.5664
```

Key learning

`Shape` cannot be instantiated directly because it contains the pure virtual function `area()`.

Concept 12: Virtual Base Class and Diamond Inheritance

Aim

To solve duplicate-base ambiguity in diamond inheritance using a virtual base class.

Program

```
#include <iostream>
#include <string>
#include <utility>

class Person {
protected:
    std::string name;

public:
    explicit Person(std::string personName) : name(std::move(personName)) {}
```

```
    void displayName() const {
        std::cout << "Name: " << name << '\n';
    }
};

class Student : virtual public Person {
public:
    Student() : Person("Unknown") {}
};

class Employee : virtual public Person {
public:
    Employee() : Person("Unknown") {}
};

class TeachingAssistant : public Student, public Employee {
public:
    explicit TeachingAssistant(std::string assistantName)
        : Person(std::move(assistantName)), Student(), Employee() {}
};

int main() {
    TeachingAssistant assistant("Riya");
    assistant.displayName();
    return 0;
}
```

Output

```
Name: Riya
```

Diagram

```
          Person
        virtual base
         /      \
```

```
   Student     Employee
      \       /
     TeachingAssistant
```

Only one Person part exists in TeachingAssistant.

Concept 13: Friend Class

Aim

To demonstrate access to private data using a friend class.

Program

```
#include <iostream>

class Account {
private:
   double balance;

   friend class Auditor;

public:
   explicit Account(double initialBalance) : balance(initialBalance) {}
};

class Auditor {
public:
   void inspect(const Account& account) const {
      std::cout << "Account Balance: " << account.balance << '\n';
   }
};

int main() {
   Account account(5000.0);
   Auditor auditor;
```

```
    auditor.inspect(account);
    return 0;
}
```

Output

```
Account Balance: 5000
```

Key learning

A friend class can access private data, but friendship should be used only when there is a clear design reason.

Concept 14: Nested Class

Aim

To create and use a nested class.

Program

```
#include <iostream>
#include <string>
#include <utility>

class University {
public:
    class Department {
    private:
        std::string name;

    public:
        explicit Department(std::string departmentName)
            : name(std::move(departmentName)) {}

        void display() const {
            std::cout << "Department: " << name << '\n';
        }
    };
```

```
};

int main() {
    University::Department department("Artificial Intelligence and Data Science");
    department.display();
    return 0;
}
```

Output

```
Department: Artificial Intelligence and Data Science
```

Concept 15: Mini-Project — Vehicle Rental System

Aim

To build an inheritance-based vehicle rental application.

Program

```
#include <iostream>
#include <string>
#include <utility>

class Vehicle {
protected:
    std::string registrationNumber;
    double ratePerDay;

public:
    Vehicle(std::string registration, double rate)
        : registrationNumber(std::move(registration)), ratePerDay(rate) {}

    virtual double calculateRent(int days) const {
        return ratePerDay * days;
    }

    virtual void display() const {
```

```
        std::cout << "Registration: " << registrationNumber << '\n';
        std::cout << "Rate per day: " << ratePerDay << '\n';
    }

    virtual ~Vehicle() = default;
};

class Car : public Vehicle {
private:
    int numberOfDoors;

public:
    Car(std::string registration, double rate, int doors)
        : Vehicle(std::move(registration), rate), numberOfDoors(doors) {}

    void display() const override {
        Vehicle::display();
        std::cout << "Doors: " << numberOfDoors << '\n';
    }
};

class Bike : public Vehicle {
private:
    int engineCapacity;

public:
    Bike(std::string registration, double rate, int capacity)
        : Vehicle(std::move(registration), rate), engineCapacity(capacity) {}

    double calculateRent(int days) const override {
        return ratePerDay * days * 0.9;
    }

    void display() const override {
```

```
        Vehicle::display();
        std::cout << "Engine Capacity: " << engineCapacity << " cc\n";
    }
};

int main() {
    Car car("MH12AB1234", 2000.0, 5);
    Bike bike("MH12CD5678", 800.0, 150);

    std::cout << "Car Details\n";
    car.display();
    std::cout << "Rent for 3 days: " << car.calculateRent(3) << "\n\n";

    std::cout << "Bike Details\n";
    bike.display();
    std::cout << "Rent for 3 days: " << bike.calculateRent(3) << '\n';

    return 0;
}
```

Expected output

```
Car Details
Registration: MH12AB1234
Rate per day: 2000
Doors: 5
Rent for 3 days: 6000

Bike Details
Registration: MH12CD5678
Rate per day: 800
Engine Capacity: 150 cc
Rent for 3 days: 2160
```

Suggested extensions

- Add Bus as another derived class.
- Add vehicle availability status.
- Use a menu-driven interface.
- Store multiple vehicles using std::vector.

Concept 16: Mini-Project — Employee Payroll System
Aim

To develop a salary system using an abstract class and derived classes.

Program

```
#include <iostream>
#include <string>
#include <utility>

class Employee {
protected:
    int employeeId;
    std::string name;

public:
    Employee(int id, std::string employeeName)
        : employeeId(id), name(std::move(employeeName)) {}

    virtual double calculateSalary() const = 0;

    void displayBasicDetails() const {
        std::cout << "Employee ID: " << employeeId << '\n';
        std::cout << "Name: " << name << '\n';
    }

    virtual ~Employee() = default;
};

class PermanentEmployee : public Employee {
```

```
private:
    double basicSalary;
    double allowance;

public:
    PermanentEmployee(int id, std::string employeeName, double basic, double extra)
        : Employee(id, std::move(employeeName)), basicSalary(basic), allowance(extra) {}

    double calculateSalary() const override {
        return basicSalary + allowance;
    }
};

class ContractEmployee : public Employee {
private:
    double hourlyRate;
    int hoursWorked;

public:
    ContractEmployee(int id, std::string employeeName, double rate, int hours)
        : Employee(id, std::move(employeeName)), hourlyRate(rate), hoursWorked(hours) {}

    double calculateSalary() const override {
        return hourlyRate * hoursWorked;
    }
};

void displayPaySlip(const Employee& employee) {
    employee.displayBasicDetails();
    std::cout << "Salary: " << employee.calculateSalary() << "\n\n";
}

int main() {
    PermanentEmployee permanentEmployee(101, "Asha", 40000.0, 8000.0);
```

```
    ContractEmployee contractEmployee(102, "Vikas", 500.0, 80);

    displayPaySlip(permanentEmployee);
    displayPaySlip(contractEmployee);

    return 0;
}
```

Expected output

```
Employee ID: 101
Name: Asha
Salary: 48000

Employee ID: 102
Name: Vikas
Salary: 40000
```

Concepts used

- Abstract class.
- Pure virtual function.
- Inheritance.
- Function overriding.
- Polymorphic interface through a base reference.
- Constructors.

Quick Reference

Basic inheritance syntax

```
class Derived : public Base {
};
```

Access rules

Base member	Direct access in derived class

public	Yes, subject to inheritance mode
protected	Yes, subject to inheritance mode
private	No

Constructor/destructor order

```
Construction: Base → Derived
Destruction: Derived → Base
```

Virtual base syntax

```
class Student : virtual public Person {
};
```

Abstract class syntax

```
class Shape {
public:
   virtual double area() const = 0;
   virtual ~Shape() = default;
};
```

Friend class syntax

```
class Account {
   friend class Auditor;
};
```

Nested class syntax

```
class Outer {
public:
   class Inner {
   };
};
```